

ASSIGNMENT 6.3

Name: P. Niharika

HT NO: 2303A51694

Batch: 24

Task Description #1 (Loops – Automorphic Numbers in a Range)

- Task: Prompt AI to generate a function that displays all Automorphic numbers between 1 and 1000 using a for loop.

- Instructions:

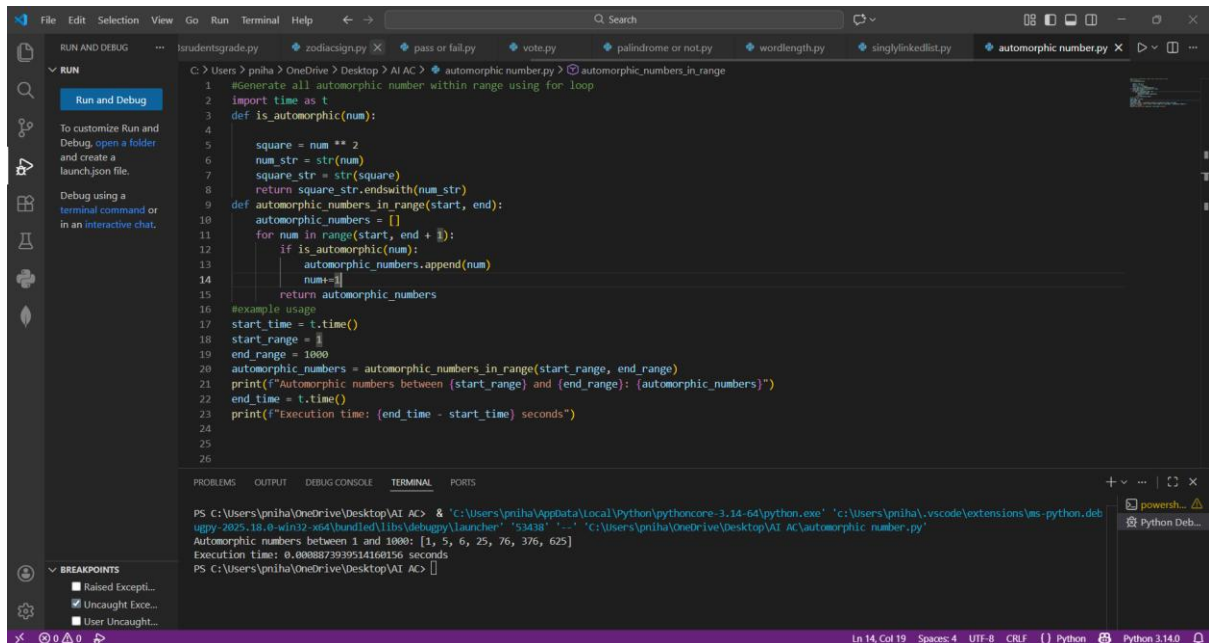
- o Get AI-generated code to list Automorphic numbers using a for loop.

Task Description #1 (Loops – Automorphic Numbers in a Range)

- Task: Prompt AI to generate a function that displays all Automorphic numbers between 1 and 1000 using a for loop.

- Instructions:

- o Get AI-generated code to list Automorphic numbers using a for loop.



```
1 #generate all automorphic number within range using for loop
2 import time as t
3 def is_automorphic(num):
4
5     square = num ** 2
6     num_str = str(num)
7     square_str = str(square)
8     return square_str.endswith(num_str)
9
10 def automorphic_numbers_in_range(start, end):
11     automorphic_numbers = []
12     for num in range(start, end + 1):
13         if is_automorphic(num):
14             automorphic_numbers.append(num)
15             num+=1
16     return automorphic_numbers
17
18 #example usage
19 start_time = t.time()
20 start_range = 1
21 end_range = 1000
22 automorphic_numbers = automorphic_numbers_in_range(start_range, end_range)
23 print(f"Automorphic numbers between {start_range} and {end_range}: {automorphic_numbers}")
24 end_time = t.time()
25 print(f"Execution time: {end_time - start_time} seconds")
26
```

PS C:\Users\pnih\OneDrive\Desktop\AI AC> & "C:\Users\pnih\AppData\Local\Python\pythoncore-3.14-64\python.exe" "C:\Users\pnih\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher" "53438" "-" "C:\Users\pnih\OneDrive\Desktop\AI AC\automorphic_number.py"

Automorphic numbers between 1 and 1000: [1, 5, 6, 25, 76, 376, 625]

Execution time: 0.0008873939514160156 seconds

PS C:\Users\pnih\OneDrive\Desktop\AI AC> []

Task Description #2 (Conditional Statements – Online Shopping Feedback

Classification)

- Task: Ask AI to write nested if-elif-else conditions to classify online shopping feedback as Positive, Neutral, or Negative based on a numerical rating (1–5).

- Instructions:

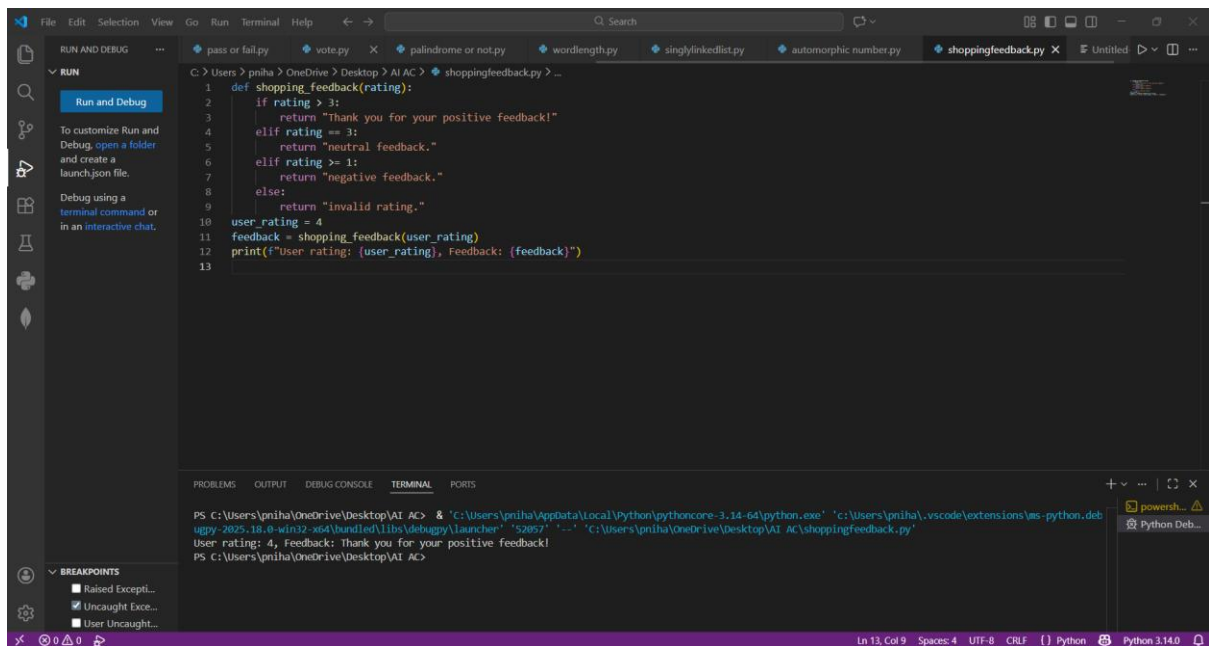
- o Generate initial code using nested if-elif-else.

- o Analyze correctness and readability.

- o Ask AI to rewrite using dictionary-based or match-case structure.

Expected Output #2:

- Feedback classification function with explanation and an alternative approach.



The screenshot shows a Visual Studio Code editor window with a Python file named `shoppingfeedback.py`. The code defines a function `shopping_feedback(rating)` that uses nested if-elif-else statements to classify feedback based on a rating from 1 to 5. The function returns "Thank you for your positive feedback!" for ratings 4 and 5, "neutral feedback." for rating 3, and "negative feedback." for ratings 1 and 2. An `else` clause returns "invalid rating." for ratings below 1. Below the function, the code sets `user_rating = 4`, calls the function, and prints the result. The terminal at the bottom shows the command to run the script, which executes successfully, outputting: `User rating: 4, Feedback: Thank you for your positive feedback!`

```
1 def shopping_feedback(rating):
2     if rating > 3:
3         return "Thank you for your positive feedback!"
4     elif rating == 3:
5         return "neutral feedback."
6     elif rating >= 1:
7         return "negative feedback."
8     else:
9         return "invalid rating."
10
11 user_rating = 4
12 feedback = shopping_feedback(user_rating)
13 print(f"User rating: {user_rating}, Feedback: {feedback}")
```

```
PS C:\Users\pnaha\OneDrive\Desktop\AI AC> & 'C:\Users\pnaha\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'C:\Users\pnaha\.vscode\extensions\ms-python.debugpy-2025.12.8-win32-x64\lib\debugpy\launcher' '52857' '-' 'C:\Users\pnaha\OneDrive\Desktop\AI AC\shoppingfeedback.py'
User rating: 4, Feedback: Thank you for your positive feedback!
PS C:\Users\pnaha\OneDrive\Desktop\AI AC>
```

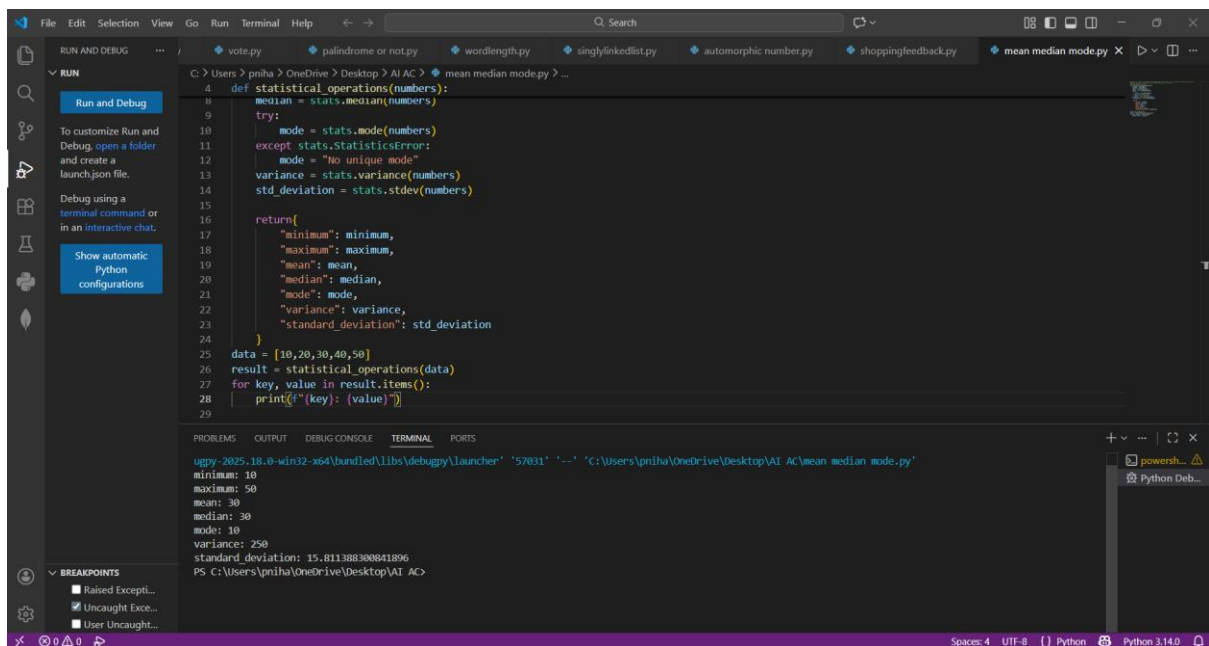
Task 3: Statistical_operations

Define a function named `statistical_operations(tuple_num)` that performs the following statistical operations on a tuple of numbers:

- Minimum, Maximum
- Mean, Median, Mode
- Variance, Standard Deviation

While writing the function, observe the code suggestions provided by GitHub Copilot. Make decisions to accept, reject, or modify the suggestions based on

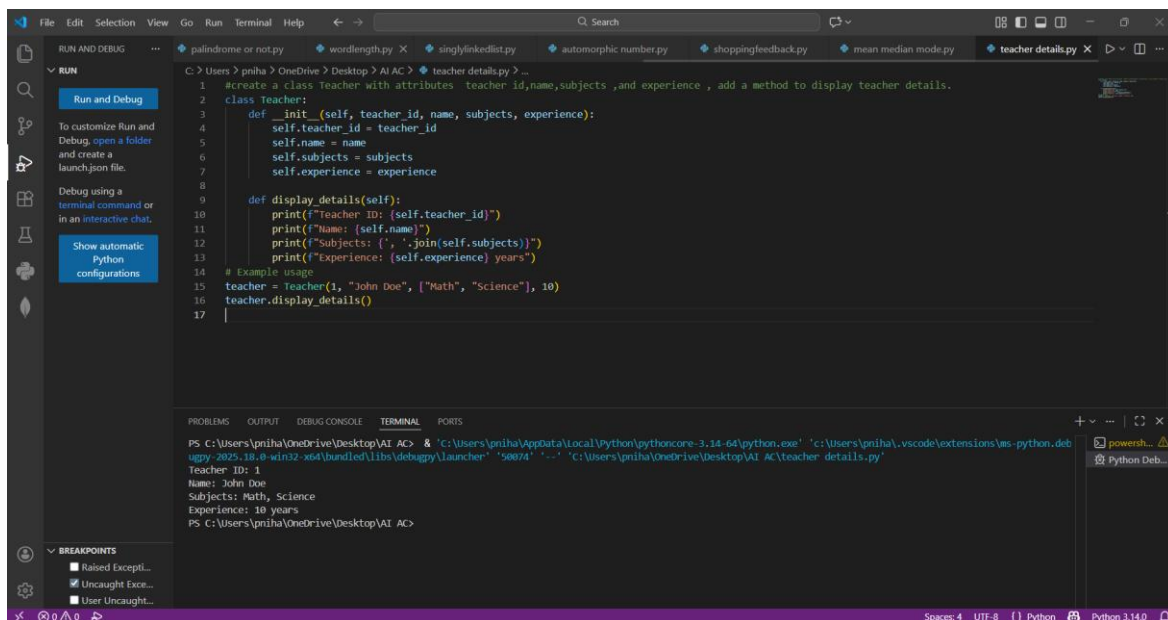
their relevance and correctness



```
File Edit Selection View Go Run Terminal Help
C:\Users\pnha> OneDrive\ Desktop > AI AC > mean median mode.py ...
4 def statistical_operations(numbers):
5     median = stats.median(numbers)
6     try:
7         mode = stats.mode(numbers)
8     except stats.StatisticsError:
9         mode = "No unique mode"
10    variance = stats.variance(numbers)
11    std_deviation = stats.stdev(numbers)
12
13    return{
14        "minimum": minimum,
15        "maximum": maximum,
16        "mean": mean,
17        "median": median,
18        "mode": mode,
19        "variance": variance,
20        "standard_deviation": std_deviation
21    }
22
23    data = [10,20,30,40,50]
24    result = statistical_operations(data)
25    for key, value in result.items():
26        print(f"{key}: {value}")
27
28
29
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
ugpy-2025-10-0-win32-x64\bundled\libs\debugpy\launcher' '57831' '--' 'C:\Users\pnha\OneDrive\Desktop\AI AC\mean median mode.py'
minimum: 10
maximum: 50
mean: 30
median: 30
mode: 10
variance: 250
standard_deviation: 15.811388340841896
PS C:\Users\pnha\OneDrive\Desktop\AI AC>
```

Task 4: Teacher Profile

- Prompt: Create a class Teacher with attributes teacher_id, name, subject, and experience. Add a method to display teacher details.
- Expected Output: Class with initializer, method, and object creation.



```
File Edit Selection View Go Run Terminal Help
C:\Users\pnha> OneDrive\ Desktop > AI AC > teacher details.py ...
1 Create a class Teacher with attributes teacher_id,name,subjects ,and experience , add a method to display teacher details.
2 class Teacher:
3     def __init__(self, teacher_id, name, subjects, experience):
4         self.teacher_id = teacher_id
5         self.name = name
6         self.subjects = subjects
7         self.experience = experience
8
9
10    def display_details(self):
11        print(f"Teacher ID: {self.teacher_id}")
12        print(f"Name: {self.name}")
13        print(f"Subjects: {', '.join(self.subjects)}")
14        print(f"Experience: {self.experience} years")
15
16    # Example usage
17    teacher = Teacher(1, "John Doe", ["Math", "Science"], 10)
18    teacher.display_details()
19
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\pnha\OneDrive\Desktop\AI AC> & 'C:\Users\pnha\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'C:\Users\pnha\.vscode\extensions\ms-python.de
ugpy-2025-10-0-win32-x64\bundled\libs\debugpy\launcher' '50874' '--' 'C:\Users\pnha\OneDrive\Desktop\AI AC\teacher details.py'
Teacher ID: 1
Name: John Doe
Subjects: Math, Science
Experience: 10 years
PS C:\Users\pnha\OneDrive\Desktop\AI AC>
```

Task #5 – Zero-Shot Prompting with Conditional Validation

Use zero-shot prompting to instruct an AI tool to generate a function that validates an Indian mobile number.

Requirements

- The function must ensure the mobile number:

- o Starts with 6, 7, 8, or 9

- o Contains exactly 10 digits

Expected Output

- A valid Python function that performs all required validations

without using any input-output examples in the prompt.

```

1 #generate a python code that validates an indian mobile number . starts with 6,7,9 or 9
2 import re
3 def validate_indian_mobile_number(mobile_number):
4     pattern = r'^[6-9]\d{9}$'
5     if re.match(pattern, mobile_number):
6         return True
7     else:
8         return False
9 # Example usage
10 mobile_number = input("Enter an Indian mobile number: ")
11 if validate_indian_mobile_number(mobile_number):
12     print("Valid Indian mobile number.")
13 else:
14     print("Invalid Indian mobile number.")
  
```

```

PS C:\Users\pnha\OneDrive\Desktop\AI AC> & 'C:\Users\pnha\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'C:\Users\pnha\vscode\extensions\ms-python.debugpy-2025.18.9-win32-x64\bundled\libs\debugpy\launcher' '53005' '-' 'C:\Users\pnha\OneDrive\Desktop\AI AC\indianmobile no.py'
Enter an Indian mobile number: 8520883781
Valid Indian mobile number.
PS C:\Users\pnha\OneDrive\Desktop\AI AC>
  
```

Task Description #6 (Loops – Armstrong Numbers in a Range)

Task: Write a function using AI that finds all Armstrong numbers in a user-specified range (e.g., 1 to 1000).

Instructions:

- Use a for loop and digit power logic.
- Validate correctness by checking known Armstrong numbers (153, 370, etc.).
- Ask AI to regenerate an optimized version (using list comprehensions).

Expected Output #7:

- Python program listing Armstrong numbers in the range.
- Optimized version with explanation.

```
C:\Users\pnih> OneDrive\Desktop> AI AC> arm.py > ...
1 #Write a python code that finds all Armstrong numbers in a user-specified range (e.g., 1 to 1000).using for loop and digit power logic
2 def is_armstrong_number(num):
3     num_str = str(num)
4     num_digits = len(num_str)
5     sum_of_powers = sum(int(digit) ** num_digits for digit in num_str)
6     return sum_of_powers == num
7
8 def armstrong_numbers_in_range(start, end):
9     armstrong_numbers = []
10    for num in range(start, end + 1):
11        if is_armstrong_number(num):
12            armstrong_numbers.append(num)
13    return armstrong_numbers
14
15 start_range = 1
16 end_range = 1000
17 result = armstrong_numbers_in_range(start_range, end_range)
18 print(f"Armstrong numbers between {start_range} and {end_range}: {result}")

PS C:\Users\pnih> OneDrive\Desktop> AI AC> & 'C:\Users\pnih\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'C:\Users\pnih\.vscode\extensions\ms-python.debugpy-2025.18.8.win32-x64\bundled\libs\debugpy\launcher' '63717' '-' 'C:\Users\pnih\OneDrive\Desktop\AI AC\arm.py'
Armstrong numbers between 1 and 1000: [1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407]
PS C:\Users\pnih\OneDrive\Desktop\AI AC>
```

Task Description #7 (Loops – Happy Numbers in a Range)

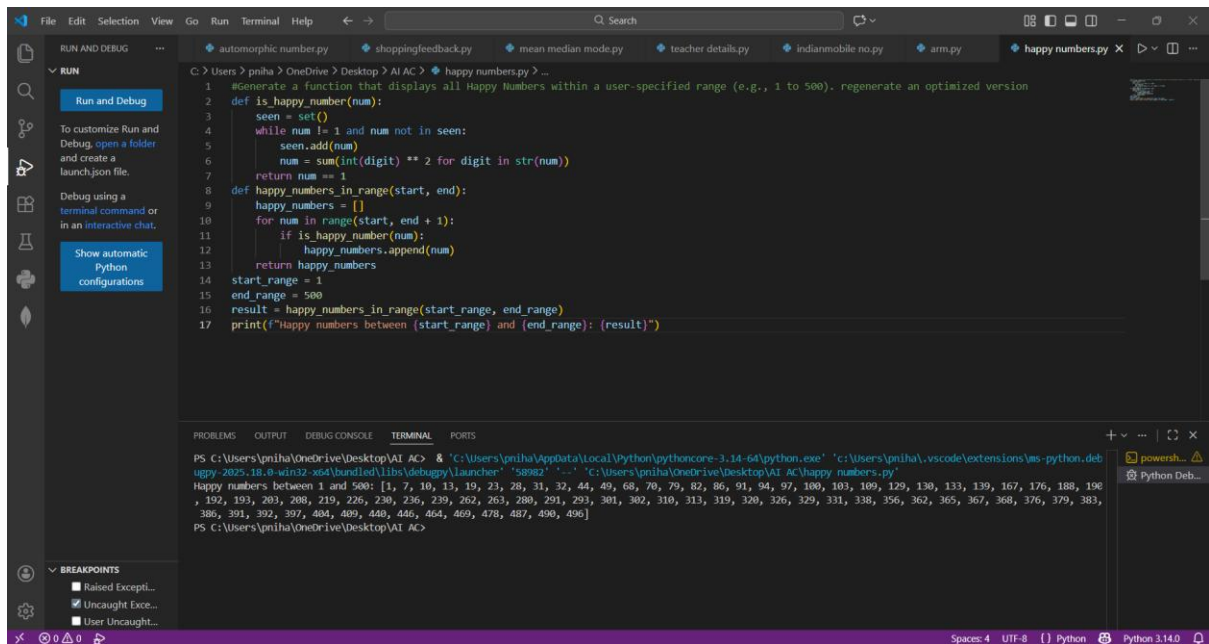
Task: Generate a function using AI that displays all Happy Numbers within a user-specified range (e.g., 1 to 500).

Instructions:

- Implement the logic using a loop: repeatedly replace a number with the sum of the squares of its digits until the result is either 1 (Happy Number) or enters a cycle (Not Happy).
- Validate correctness by checking known Happy Numbers (e.g., 1, 7, 10, 13, 19, 23, 28...).
- Ask AI to regenerate an optimized version (e.g., by using a set to detect cycles instead of infinite loops).

Expected Output #8:

- Python program that prints all Happy Numbers within a range.
- Optimized version using cycle detection with explanation.



```
1 #generate a function that displays all Happy Numbers within a user-specified range (e.g., 1 to 500). regenerate an optimized version
2 def is_happy_number(num):
3     seen = set()
4     while num != 1 and num not in seen:
5         seen.add(num)
6         num = sum(int(digit) ** 2 for digit in str(num))
7     return num == 1
8 def happy_numbers_in_range(start, end):
9     happy_numbers = []
10    for num in range(start, end + 1):
11        if is_happy_number(num):
12            happy_numbers.append(num)
13    return happy_numbers
14 start_range = 1
15 end_range = 500
16 result = happy_numbers_in_range(start_range, end_range)
17 print(f"Happy numbers between {start_range} and {end_range}: {result}")
```

PS C:\Users\pnha\OneDrive\Desktop\AI AC> & "C:\Users\pnha\AppData\Local\Python\pythoncore-3.14-64\python.exe" "C:\Users\pnha\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\libs\debugpy\launcher" "50902" "-." "C:\Users\pnha\OneDrive\Desktop\AI AC\happy_numbers.py"

Happy numbers between 1 and 500: [1, 7, 10, 13, 19, 23, 28, 31, 32, 44, 49, 68, 70, 79, 82, 86, 91, 94, 97, 100, 103, 109, 129, 130, 133, 139, 167, 176, 188, 198, 192, 193, 203, 208, 219, 226, 230, 236, 239, 262, 263, 280, 291, 293, 301, 302, 318, 313, 319, 320, 326, 329, 331, 338, 356, 362, 365, 367, 368, 376, 379, 383, 386, 391, 392, 397, 404, 409, 440, 446, 464, 469, 478, 487, 490, 496]

PS C:\Users\pnha\OneDrive\Desktop\AI AC>

Task Description #8 (Loops – Strong Numbers in a Range)

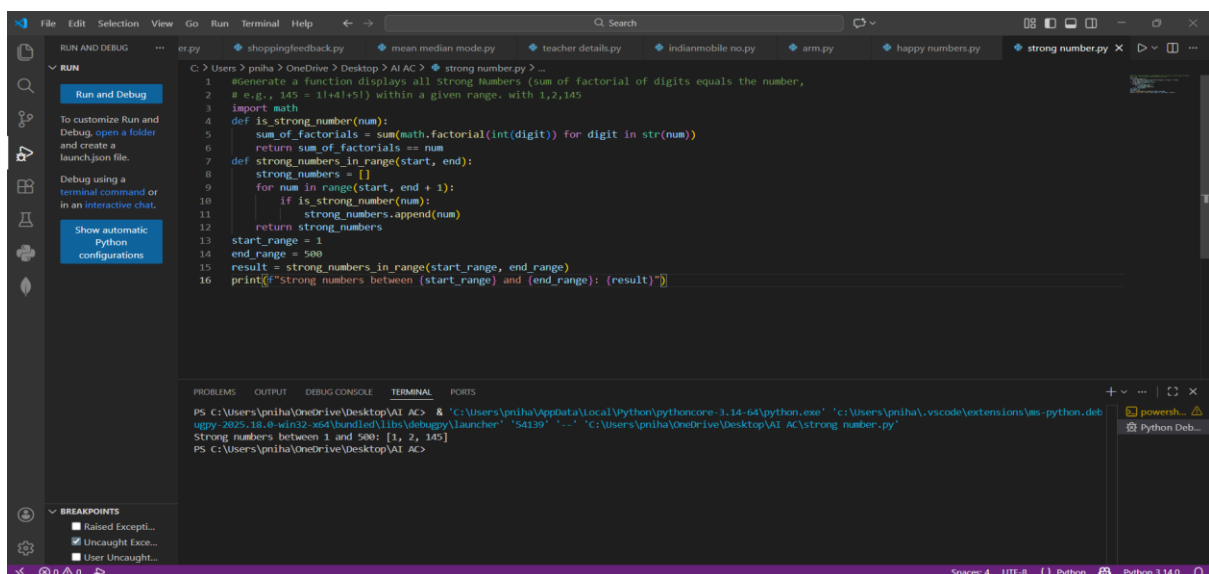
Task: Generate a function using AI that displays all Strong Numbers (sum of factorial of digits equals the number, e.g., $145 = 1! + 4! + 5!$) within a given range.

Instructions:

- Use loops to extract digits and calculate factorials.
- Validate with examples (1, 2, 145).
- Ask AI to regenerate an optimized version (precompute digit factorials).

Expected Output #9:

- Python program that lists Strong Numbers.
- Optimized version with explanation.



```
1 #generate a function displays all Strong Numbers (sum of factorial of digits equals the number,
2 # e.g., 145 = 1!+4!+5!) within a given range. with 1,2,145
3 import math
4 def is_strong_number(num):
5     sum_of_factorials = sum(math.factorial(int(digit)) for digit in str(num))
6     return sum_of_factorials == num
7 def strong_numbers_in_range(start, end):
8     strong_numbers = []
9     for num in range(start, end + 1):
10        if is_strong_number(num):
11            strong_numbers.append(num)
12    return strong_numbers
13 start_range = 1
14 end_range = 500
15 result = strong_numbers_in_range(start_range, end_range)
16 print(f"Strong numbers between {start_range} and {end_range}: {result}")
```

PS C:\Users\pnha\OneDrive\Desktop\AI AC> & "C:\Users\pnha\AppData\Local\Python\pythoncore-3.14-64\python.exe" "C:\Users\pnha\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\libs\debugpy\launcher" "54139" "-." "C:\Users\pnha\OneDrive\Desktop\AI AC\strong_number.py"

Strong numbers between 1 and 500: [1, 2, 145]

PS C:\Users\pnha\OneDrive\Desktop\AI AC>

Task #9 – Few-Shot Prompting for Nested Dictionary Extraction

Objective

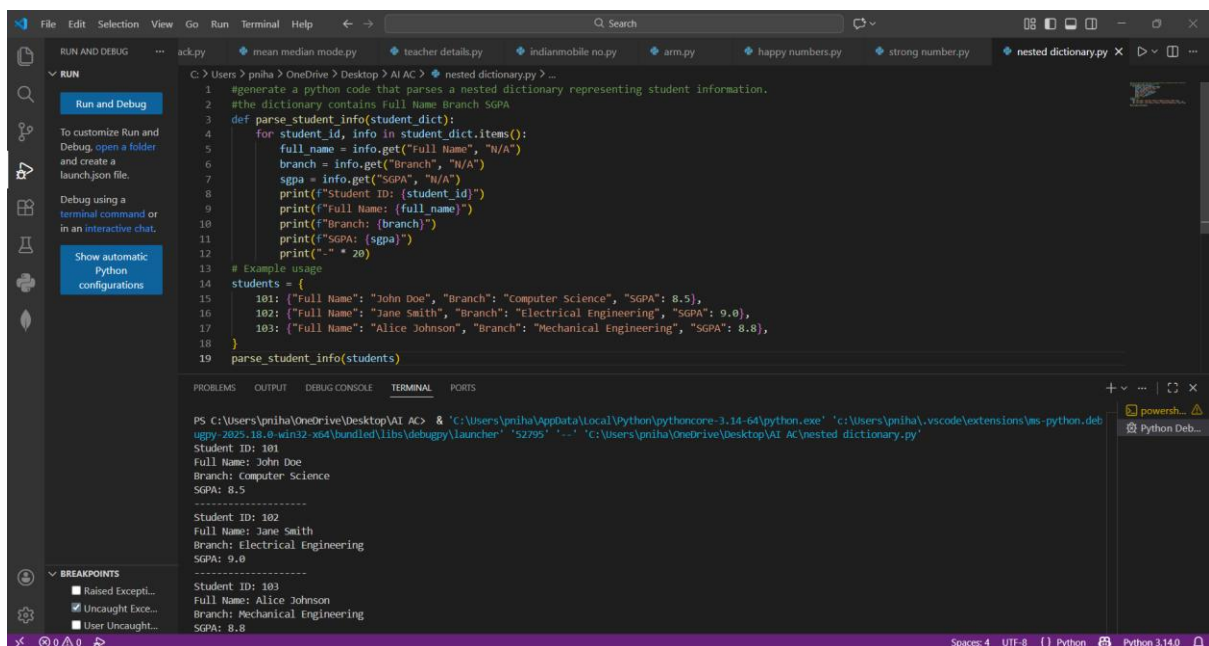
Use few-shot prompting (2–3 examples) to instruct the AI to create a function that parses a nested dictionary representing student information.

Requirements

- The function should extract and return:
 - o Full Name
 - o Branch
 - o SGPA

Expected Output

A reusable Python function that correctly navigates and extracts values from nested dictionaries based on the provided examples



```
File Edit Selection View Go Run Terminal Help
C:\Users\priha> OneDrive Desktop > AI AC > nested dictionary.py > ...
1 #generate a python code that parses a nested dictionary representing student information.
2 #the dictionary contains Full Name Branch SGPA
3 def parse_student_info(student_dict):
4     for student_id, info in student_dict.items():
5         full_name = info.get("Full Name", "N/A")
6         branch = info.get("Branch", "N/A")
7         sgpa = info.get("SGPA", "N/A")
8         print(f"Student ID: {student_id}")
9         print(f"Full Name: {full_name}")
10        print(f"Branch: {branch}")
11        print(f"SGPA: {sgpa}")
12        print("-" * 20)
13 # Example usage
14 students = {
15     101: {"Full Name": "John Doe", "Branch": "Computer Science", "SGPA": 8.5},
16     102: {"Full Name": "Jane Smith", "Branch": "Electrical Engineering", "SGPA": 9.0},
17     103: {"Full Name": "Alice Johnson", "Branch": "Mechanical Engineering", "SGPA": 8.8},
18 }
19 parse_student_info(students)
```

```
PS C:\Users\priha\OneDrive\Desktop\AI AC> & "C:\Users\priha\AppData\Local\Python\pythoncore-3.14-64\python.exe" "C:\Users\priha\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\libs\debugpy\launcher" "52795" "..." "C:\Users\priha\OneDrive\Desktop\AI AC\nested dictionary.py"
Student ID: 101
Full Name: John Doe
Branch: Computer Science
SGPA: 8.5
-----
Student ID: 102
Full Name: Jane Smith
Branch: Electrical Engineering
SGPA: 9.0
-----
Student ID: 103
Full Name: Alice Johnson
Branch: Mechanical Engineering
SGPA: 8.8
```

Task Description #10 (Loops – Perfect Numbers in a Range)

Task: Generate a function using AI that displays all Perfect Numbers within a user-specified range (e.g., 1 to 1000).

Instructions:

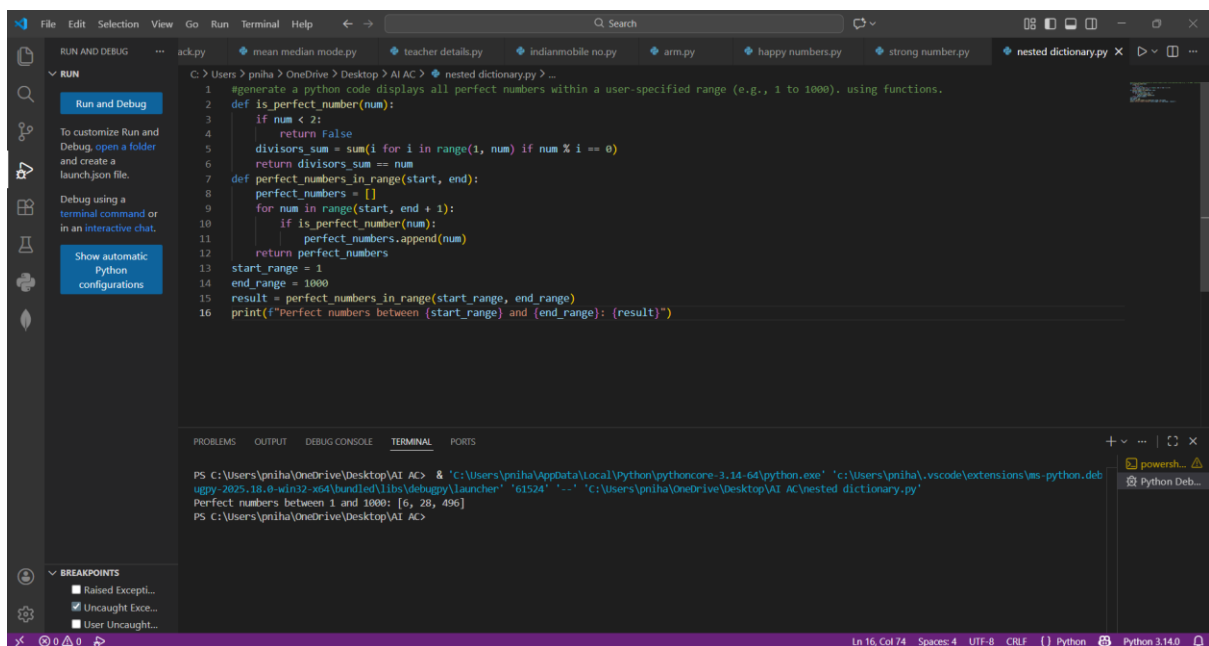
- A Perfect Number is a positive integer equal to the sum of its proper divisors (excluding itself).

o Example: $6 = 1 + 2 + 3$, $28 = 1 + 2 + 4 + 7 + 14$.

- Use a for loop to find divisors of each number in the range.
- Validate correctness with known Perfect Numbers (6, 28, 496...).
- Ask AI to regenerate an optimized version (using divisor check only up to $\sqrt{n}$).

Expected Output #12:

- Python program that lists Perfect Numbers in the given range.
- Optimized version with explanation.



The screenshot shows a Visual Studio Code editor with a Python file named `nested dictionary.py`. The code defines a function `is_perfect_number(num)` that checks if a number is perfect by summing its divisors. It then uses this function to find all perfect numbers in a specified range (1 to 1000). The terminal output shows the execution of the program, which prints the perfect numbers between 1 and 1000: [6, 28, 496].

```
1 #generate a python code displays all perfect numbers within a user-specified range (e.g., 1 to 1000). using functions.
2 def is_perfect_number(num):
3     if num < 2:
4         return False
5     divisors_sum = sum(i for i in range(1, num) if num % i == 0)
6     return divisors_sum == num
7 def perfect_numbers_in_range(start, end):
8     perfect_numbers = []
9     for num in range(start, end + 1):
10         if is_perfect_number(num):
11             perfect_numbers.append(num)
12     return perfect_numbers
13 start_range = 1
14 end_range = 1000
15 result = perfect_numbers_in_range(start_range, end_range)
16 print(f"Perfect numbers between {start_range} and {end_range}: {result}")
```

Terminal Output:

```
PS C:\Users\pnaha\OneDrive\Desktop\AI AC> & "C:\Users\pnaha\AppData\Local\Python\pythoncore-3.14-64\python.exe" "c:\Users\pnaha\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher" "61524" "--" "C:\Users\pnaha\OneDrive\Desktop\AI AC\nested dictionary.py"
Perfect numbers between 1 and 1000: [6, 28, 496]
PS C:\Users\pnaha\OneDrive\Desktop\AI AC>
```